

```

----- ARCHIVO firebase.js
-----
// Configuración de Firebase
// Importante las versiones para 'firebase-app.js' y para 'firebase-firestore.js'
en ambos casos debes ser las mismas
import { initializeApp } from
"https://www.gstatic.com/firebasejs/10.13.0/firebase-app.js";
import {
  getFirestore,
  collection,
  addDoc,
  getDoc,
  getDocs,
  deleteDoc,
  doc,
  updateDoc,
} from "https://www.gstatic.com/firebasejs/10.13.0/firebase-database.js";

// Firebase configuration (replace with your actual project details)
const firebaseConfig = {
  apiKey: "AIzaSyAfpEDzd8wc6t9Y3foI2HDrWVL_MIzhYnA",
  authDomain: "asistencia-93328.firebaseio.com",
  databaseURL: "https://asistencia-93328-default-rtdb.firebaseio.com",
  projectId: "asistencia-93328",
  storageBucket: "asistencia-93328.firebaseio.com",
  messagingSenderId: "692275978617",
  appId: "1:692275978617:web:5579ef6c0aeb2d58c7cfa8"
};

// Initialize Firebase
export const app = initializeApp(firebaseConfig);
// Inicializa Cloud Firestore y obtén una referencia al servicio.
export const db = getFirestore(app);

const coleccion = "cursos";

// Esta función agrega un nuevo Curso a una colección en Firestore, con los
detalles proporcionados como nombre, cédula, edad, sexo, teléfono y cargo.
export const addCurso = (curso, ubicacion, capacidad, obs) =>
  addDoc(collection(db, coleccion), { curso, ubicacion, capacidad, obs });

// Esta función obtiene todos los documentos de una colección de Cursos en
Firestore y devuelve una promesa que se resuelve con los datos de esos documentos.
export const getCursosCollection = () => getDocs(collection(db, coleccion));

// Esta función obtiene un documento específico de una colección de Cursos en
Firestore utilizando su ID como referencia y devuelve una promesa que se resuelve
con los datos del documento.
export const getCursoCollection = (id) => {
  const docRef = doc(db, coleccion, id);

```

```
    return getDoc(docRef);
};
```

// Esta función actualiza un documento específico en una colección de Cursos en Firestore con los nuevos campos proporcionados, utilizando el ID del documento como referencia.

```
export const updateCursoCollection = (id, newFields) =>
    updateDoc(doc(db, coleccion, id), newFields);
```

// Esta función elimina un documento específico de una colección de Cursos en Firestore utilizando su ID como referencia.

```
export const deleteCursoCollection = (id) => deleteDoc(doc(db, coleccion, id));
```

----- ARCHIVO todoCrud.js

```
import {
    addCurso,
    getCursosCollection,
    deleteCursoCollection,
    getCursoCollection,
    updateCursoCollection,
} from "../firebase.js";
```

```
/**
```

```
 * Función para levantar Venta Modal
```

```
 */
```

```
window.miModal = async function (idModal, idCurso = "") {
```

```
    try {
```

```
        await validarModal(idModal);
```

```
        let url = "";
```

```
        switch (idModal) {
```

```
            case "agregarCursoModal":
```

```
                url = "modales/modalAdd.php";
```

```
                break;
```

```
            case "detalleCursoModal":
```

```
                url = "modales/modalDetalles.php";
```

```
                break;
```

```
            case "editarCursoModal":
```

```
                url = "modales/modalEditar.php";
```

```
                break;
```

```
            case "eliminarCursoModal":
```

```
                url = "modales/modalDelete.php";
```

```
                break;
```

```
            default:
```

```
                throw new Error(`El idModal '${idModal}' no es válido`);
```

```
        }
```

```

const response = await fetch(url);

if (!response.ok) {
  throw new Error("Error al cargar la modal");
}

// response.text() es un método en programación que se utiliza para obtener el
contenido de texto de una respuesta HTTP
const data = await response.text();

// Crear un elemento div para almacenar el contenido de la modal
const modalContainer = document.createElement("div");
modalContainer.innerHTML = data;

// Agregar la modal al documento actual
document.body.appendChild(modalContainer);

// Mostrar la modal
const modalElement = modalContainer.querySelector(`#${idModal}`);
const myModal = new bootstrap.Modal(modalElement);
myModal.show();

if (idModal === "detalleCursoModal") {
  await cargarDetalleCurso(idCurso);
} else if (idModal === "editarCursoModal") {
  await getCursoUpdateCollection(idCurso);
} else if (idModal === "eliminarCursoModal") {
  let DeleteBtn = document.querySelector("#confirmDeleteBtn");
  DeleteBtn.addEventListener("click", async () => {
    await eliminarCurso(idCurso);
  });
}
} catch (error) {
  console.error(error);
}
};

//Función para validar si existe una modal abierta
async function validarModal(idModal) {
  const existingModal = document.getElementById(idModal);
  if (existingModal) {
    const modal = bootstrap.Modal.getInstance(existingModal);
    if (modal) {
      modal.hide();
    }
    existingModal.remove();
  }
}

/**

```

```

* Función para obtener todas las colecciones
*/
async function mostrarCursosEnHTML() {
  try {
    const CursosCollection = getCursosCollection();
    const queryCollection = await CursosCollection;

    const tablaCursos = document.querySelector("#tablaCursos tbody");
    tablaCursos.innerHTML = "";

    queryCollection.forEach((doc) => {
      const Curso = doc.data();
      const fila = document.createElement("tr");
      fila.id = doc.id; // Asignar el ID del documento al elemento tr
      fila.innerHTML = `
        <td>${Curso.nombre}</td>
        <td>${Curso.edad}</td>
        <td>${Curso.cedula}</td>
        <td>${Curso.sexo}</td>
        <td>${Curso.cargo}</td>
        <td>${Curso.telefono}</td>
        <td>
          <a title="Ver detalles del Curso" href="#"
onclick="window.miModal('detalleCursoModal', '${doc.id}')" class="btn btn-success">
            <i class="bi bi-binoculars"></i>
          </a>
          <a title="Editar datos del Curso" href="#"
onclick="window.miModal('editarCursoModal', '${doc.id}')" class="btn btn-warning">
            <i class="bi bi-pencil-square"></i>
          </a>
          <a title="Eliminar datos del Curso" href="#"
onclick="window.miModal('eliminarCursoModal', '${doc.id}')" class="btn btn-danger">
            <i class="bi bi-trash"></i>
          </a>
        </td>
      `;
      tablaCursos.appendChild(fila);
      //console.log("Curso:", doc.data());
    });
  } catch (error) {
    console.error("Error al obtener los Cursos:", error);
  }
}

window.addEventListener("DOMContentLoaded", mostrarCursosEnHTML);

/**
* Función para agregar un nuevo Curso
*/
window.addNuevoCurso = async function (event) {

```

```

event.preventDefault();

const formulario = document.querySelector("#formularioCurso");
const formData = new FormData(formulario);

// Convertir FormData a un objeto JSON
const formDataJSON = {};
formData.forEach((value, key) => {
  formDataJSON[key] = value;
});

const { nombre, cedula, edad, sexo, telefono, cargo } = formDataJSON;

try {
  await addCurso(nombre, cedula, edad, sexo, telefono, cargo);
  formulario.reset();
  setTimeout(() => {
    $("#agregarCursoModal").css("opacity", "");
    $("#agregarCursoModal").modal("hide");
  }, 300);

  window.mostrarAlerta({ tipoToast: "success", mensaje: "¡Curso registrado correctamente!" });
} catch (error) {
  console.log(error);
}
};

/**
 * Función para cargar y mostrar los detalles del Curso en la modal
 */
async function cargarDetalleCurso(id) {
  try {
    const CursoDoc = await getCursoCollection(id);
    if (CursoDoc.exists()) {
      const CursoData = CursoDoc.data();
      const { curso, ubicacion, capacidad, obs } = CursoData;

      const ulDetalleCurso = document.querySelector("#detalleCursoContenido ul");

      ulDetalleCurso.innerHTML = `
        <li class="list-group-item"><b>Curso:</b>
          ${curso ? curso : "No disponible"}
        </li>
        <li class="list-group-item"><b>Ubicación:</b>
          ${ubicacion ? ubicacion : "No disponible"}
        </li>
        <li class="list-group-item"><b>Capacidad:</b>
          ${capacidad ? capacidad : "No disponible"}
        </li>
      `
    }
  }
}

```

```

        <li class="list-group-item"><b>Obs:</b>
        ${obs ? obs : "No disponible"}
    </li>
    `;
} else {
    console.log("No se encontró ningún Curso con el ID:", id);
}
} catch (error) {
    console.error("Error al mostrar detalles del Curso", error);
}
}

/**
 * Buscar Curso a editar
 */
async function getCursoUpdateCollection(id) {
    try {
        const CursoDoc = await getCursoCollection(id);
        if (CursoDoc.exists()) {
            const CursoData = CursoDoc.data();
            const { curso, ubicacion, capacidad, obs } = CursoData;
            document.querySelector("#idCurso").value = id;
            document.querySelector("#curso").value = curso;
            document.querySelector("#ubicacion").value = ubicacion;
            document.querySelector("#capacidad").value = capacidad;
            document.querySelector("#obs").value = obs;
        } else {
            console.log("No se encontró ningún Curso con el ID:", id);
        }
    } catch (error) {
        console.error("Error al obtener los detalles del Curso:", error);
    }
}

/**
 * Función para actualizar el Curso
 */
window.actualizarCurso = async function (event) {
    event.preventDefault();
    const formulario = document.querySelector("#formularioCursoEdit");
    const formData = new FormData(formulario);

    // Convertir FormData a un objeto JSON
    const formDataJSON = {};
    formData.forEach((value, key) => {
        //console.log(key, value);
        formDataJSON[key] = value;
    });

    const { idCurso, curso, ubicacion, capacidad, obs } = formDataJSON;

```

```

    try {
        await updateCursoCollection(idCurso, { curso, ubicacion, capacidad, obs});
        formulario.reset();
        setTimeout(() => {
            $("#editarCursoModal").css("opacity", "");
            $("#editarCursoModal").modal("hide");
        }, 300);

        window.mostrarAlerta({ tipoToast: "success", mensaje: "¡Curso actualizado correctamente!" });
    } catch (error) {
        console.log(error);
    }
};

/**
 * Función para borrar un Curso, una colleccion
 */
async function eliminarCurso(id) {
    try {
        await deleteCursoCollection(id);
        document.querySelector(`#${id}`).remove();
        mostrarAlerta({ tipoToast: "success", mensaje: "Curso eliminado correctamente"
    });
    } catch (error) {
        console.error("Error al borrar el Curso:", error);
        mostrarAlerta({ tipoToast: "error", mensaje: "Error al eliminar el Curso" });
    }
}

/**
 * Función para mostrar alertas
 */
iziToast.settings({
    timeout: 10000,
    resetOnHover: true,
    // icon: '', // icon class
    transitionIn: "flipInX",
    transitionOut: "flipOutX",
    position: "topRight", // bottomRight, bottomLeft, topRight, topLeft, topCenter,
    bottomCenter, center
    onOpening: function () {
        console.log("Alerta abierta!");
    },
    onClosing: function () {
        console.log("Alerta cerrada!");
    },
});

window.mostrarAlerta = function ({ tipoToast, mensaje }) {
    if (tipoToast == "success") {

```

```
iziToast.success({
  timeout: 5000,
  icon: tipoToast == "success" ? "bi bi-check-circle-fill" : "bi
bi-x-circle-fill",
  title: tipoToast == "success" ? "¡Éxito!" : "¡Error!",
  message: mensaje,
});
} else if (tipoToast == "warning") {
  iziToast.success({
    timeout: 5000,
    icon: tipo == "success" ? "bi bi-check-circle-fill" : "bi bi-x-circle-fill",
    title: tipo == "success" ? "¡Éxito!" : "¡Error!",
    message: mensaje,
  });
}
};
```